



COURSEWORK SPECIFICATION

COMM040 - Text Mining and Natural Language Processing

Module Leader: Rodrigo Wilkens

Academic Year: 2025/26

Language Models

This workshop focuses on Transformer-based language models, exploring how different architectures encode meaning, context and task-specific behaviour. The goal is to contrast encoder-only, decoder-only, and open LLM APIs, analysing their assumptions, limitations, and practical usage.

Environment Setup

1. Install required libraries
 - transformers datasets torch scikit-learn
-

BERT as a Classifier

1. Load a pretrained BERT-based sentiment classifier.
2. Apply it to at least three sentences of your choice.
3. Inspect:
 - predicted label
 - confidence score

```
initial code:
from transformers import pipeline

classifier = pipeline(
    "sentiment-analysis",
    model="distilbert-base-uncased-finetuned-sst-2-english"
)

Sentences = ["This lecture was clear and well structured.",
             "The model output is fluent but misleading"]
```

- What kind of output does this model produce?
 - Is BERT generating language here?
 - What type of supervision was required to train this model?
-

BERT as an Embedding Model

1. Load a pretrained BERT encoder.
2. Tokenise a small set of short documents.
3. Extract hidden representations.
4. Filter the CLS embedding.

5. Compute similarity between documents.

```
initial code:
from transformers import AutoTokenizer, AutoModel
import torch
from sklearn.metrics.pairwise import cosine_similarity

tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
model = AutoModel.from_pretrained("bert-base-uncased")
model.eval()

texts = [
    "Transformers rely on self-attention mechanisms.",
    "BERT is an encoder-based transformer model.",
    "Language models predict the next token."
]

# pt = pytorch; tf = tensorflow
inputs = tokenizer(texts, padding=True, truncation=True, return_tensors="pt")

with torch.no_grad():
    outputs = model(**inputs) # output from the model (forward step)
```

- Which documents are most similar?
- How does this differ from word-level embeddings?
- What is the difference between an average pooling and the CLS?

Fine-tuning BERT

1. Read the script below carefully before running it.
- Save the checkpoint in a folder named “results”. Everytime you run your code it must overwrite the folder’s content
 - Train for 1 epoch
 - Set the learning rate to 2e-5 and do not use weight decay and warmup steps
 - Log every step

```
initial code:
from transformers import AutoModelForSequenceClassification, Trainer,
TrainingArguments
from datasets import Dataset

texts = [
    "I loved this movie.",
    "This was a terrible film.",
    "An excellent and engaging story.",
    "The plot was boring and predictable."
]
```

```

labels = [1, 0, 1, 0]
dataset = Dataset.from_dict({"text": texts, "label": labels})

def tokenize(batch):
    return tokenizer(batch["text"], truncation=True, padding=True)

dataset = dataset.map(tokenize, batched=True)
dataset.set_format("torch", columns=["input_ids", "attention_mask", "label"])

model = AutoModelForSequenceClassification.from_pretrained(
    "bert-base-uncased", num_labels=2
)

training_args = TrainingArguments(
    # insert your training arguments here
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=dataset
)

trainer.train()

```

GPT-style Transformer

1. Load a small decoder-only Transformer.
2. Generate text from a short prompt.
3. Modify the prompt and observe changes.

```

Initial code
from transformers import AutoTokenizer, AutoModelForCausalLM

tokenizer = AutoTokenizer.from_pretrained("gpt2")
model = AutoModelForCausalLM.from_pretrained("gpt2")
model.eval()

prompt = "The government announced that"
inputs = tokenizer(prompt, return_tensors="pt")

output = model.generate(
    **inputs,
    max_new_tokens=30,
    do_sample=True,
    temperature=0.8
)

tokenizer.decode(output[0], skip_special_tokens=True)

```

- How does this output differ from BERT's?
 - How does the temperature impacts in the output?
-

Ollama

- Replicate the exercise above

Initial code

```
import requests

response = requests.post(
    "http://localhost:11434/api/generate",
    json={
        "model": YOUR_MODEL_NAME,
        "prompt": "Explain transformers in simple terms.",
        "stream": False
    }
)

print(response.json()["response"])
```